Name: Adnan Khan **Roll No.2401047** **Branch:AI&DS** **Class:T12**
Subject:ASD&D

Experiment 03 Aim: To deploy multi-service applications using Docker Compose.

This experiment maps to LO4:

LO4: Containerization, orchestrate and deploy applications using Docker, Compose, Kubernetes and Cloud infrastructure tools.

What is Docker Compose?

Docker Compose is a powerful orchestration tool designed to simplify the management of multi-container applications. Instead of manually handling individual containers, Docker Compose allows you to define an entire stack—such as a web server, a backend service, and a database—within a single, declarative configuration file known as `docker-compose.yml`.

By acting as a single point of truth, this tool automates the orchestration of complex environments, ensuring all components work together seamlessly.

The Role of the `docker-compose.yml` File

This file serves as the blueprint for your infrastructure. It specifies several critical aspects of your application stack:

Services Defines the individual containers that make up your app.
Images: Specifies the source images used to build those containers.
Networking: Automatically creates isolated networks, allowing containers to discover and communicate with one another securely.
Configuration: Manages environment variables and port mapping (defining how traffic flows between your host machine and the containers).
Volumes: Ensures data persistence by mounting host directories into the containers, preventing data loss when containers are stopped or removed.

Core Benefits

Simplified Operations: Orchestrate multiple services with singular commands.
Consistency: Eliminates the "it works on my machine" problem by ensuring every developer runs the exact same environment configuration.
Efficiency: Streamlines the deployment process, significantly reducing the setup time for complex architectures.
Persistence: Built-in volume management makes safeguarding important database and application files

straightforward.

Essential Docker Compose Commands

Once your application is defined, you can manage the entire lifecycle of your project using the following command-line interface (CLI) operations:

Command	**Purpose**
docker compose up	Creates and starts services
docker compose up -d	Runs services in background mode
docker compose ps	Displays running containers
docker compose logs	Shows service logs
docker compose stop	Stops services
docker compose start	Starts stopped services
docker compose restart	Restarts containers
docker compose down	Removes containers and networks
docker compose down -v	Removes containers and volumes
docker compose images	Displays service images
docker compose config	Validates YAML configuration

By centralizing the control of your application components, Docker Compose transforms complex microservice architectures into manageable, replicable workflows

Step 1: Create Project Directory

Open Command Prompt and execute:

mkdir compose-

```
C:\Users\Lenovo>mkdir compose--demo

C:\Users\Lenovo>cd compose--demo
```

demo cd compose-
demo

Step 2: Create Docker Compose File

Create a file

named: docker-

compose.yml

Add the following configuration:

version: '3.8'

services:

```
  web:
    image:                  nginx
    container_name:
    webserver ports:
      - "8081:80"

  database:
    image:                mysql:8.0
```

container_name:

mysqlserver

```
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE:
      studentdb           MYSQL_USER:
      student    MYSQL_PASSWORD:
      student123

    ports:
      - "3307:3306"

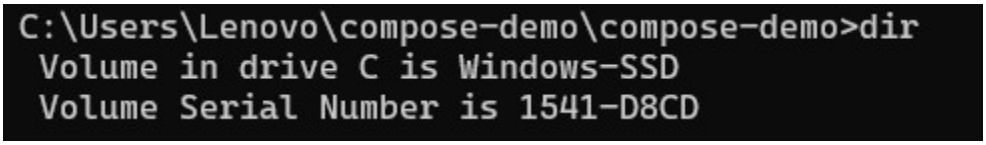
    volumes:
      - mysql-data:/var/lib/mysql

volumes:
  mysql-data:
```

Step 3: Verify Compose File

Command:

dir



Step 4: Start Multi-Service Application

Command:

docker compose up -d

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose up -d
[+] Running 2/3
 - Network compose-demo_default  Created
    0.9s
 ✔Container mysqlserver         Started
    0.9s
 ✔Container webserver           Started
    0.7s
```

Step 5: Check Running Services

Command:

docker compose ps

```
C:\Users\Lenovo\compose-demo\compose-demo>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Step 6: Access Web Service

Open a browser and enter:



http://localhost:8081

The Nginx web server is successfully deployed.

Step 7: View Container Logs

Command:

docker compose logs database

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose logs database
mysqlserver  | 2026-07-28 05:06:21+00:00 [Note] [Entrypoint]: Entrypoint
 script for MySQL Server 8.0.46-1.el9 started.
mysqlserver  | 2026-07-28 05:06:22+00:00 [Note] [Entrypoint]: Switching
to dedicated user 'mysql'
mysqlserver  | 2026-07-28 05:06:22+00:00 [Note] [Entrypoint]: Entrypoint
 script for MySQL Server 8.0.46-1.el9 started.
mysqlserver  | 2026-07-28 05:06:22+00:00 [Note] [Entrypoint]: Initializi
ng database files
mysqlserver  | 2026-07-28T05:06:22.283058Z 0 [Warning] [MY-011068] [Serv
er] The syntax '--skip-host-cache' is deprecated and will be removed in
a future release. Please use SET GLOBAL host_cache_size=0 instead.
mysqlserver  | 2026-07-28T05:06:22.283617Z 0 [System] [MY-013169] [Serve
r] /usr/sbin/mysqld (mysqld 8.0.46) initializing of server in progress a
s process 81
mysqlserver  | 2026-07-28T05:06:22.302075Z 1 [System] [MY-013576] [InnoD
B] InnoDB initialization has started.
mysqlserver  | 2026-07-28T05:06:23.692945Z 1 [System] [MY-013577] [InnoD
B] InnoDB initialization has ended.
mysqlserver  | 2026-07-28T05:06:26.580115Z 6 [Warning] [MY-010453] [Serv
er] root@localhost is created with an empty password ! Please consider s
witching off the --initialize-insecure option.
mysqlserver  | 2026-07-28 05:06:34+00:00 [Note] [Entrypoint]: Database f
iles initialized
mysqlserver  | 2026-07-28 05:06:34+00:00 [Note] [Entrypoint]: Starting t
emporary server
```

Step 8: Verify Docker Images

Command:

docker compose images

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose images
CONTAINER           REPOSITORY          TAG                 IMAGE ID
        SIZE
mysqlserver         mysql               8.0                 6cd09145362d
        799MB
webserver           nginx               latest              4e5db4761e0f
        161MB
```

Step 9: Verify Docker Volume

Command:

docker volume ls

```
C:\Users\Lenovo\compose-demo\compose-demo>docker volume ls
DRIVER    VOLUME NAME
local     compose-demo_mysql-data
```

The volume provides persistent storage for MySQL data.

Step 10: Stop Application

Command:

docker compose down

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose down
[+] Running 3/3
 ✔Container webserver            Removed
 ✔Container mysqlserver          Remove...
 ✔Network compose-demo_default   Removed
```

This stops and removes the containers.

Conclusion
Key Concepts Mastered

- **Multi-Container Orchestration:** Successfully deployed and managed a multi-service application architecture, transitioning from single-container management to full-stack orchestration.
- **Infrastructure as Code:** Gained practical experience designing a `docker-compose.yml` file to define an entire application environment in a single, version-controllable document.

Practical Skills Acquired

- **Service Configuration:** Configured multiple interacting services, specifying Docker images, mapping ports, and defining environment variables for each component.
- **Data Persistence:** Implemented Docker volumes to securely store database files and application data, ensuring information survives container restarts and removals.
- **Lifecycle Management:** Utilized standard Docker Compose commands to efficiently start, monitor, and tear down the entire application stack simultaneously.

Final Takeaway

This experiment highlighted the operational value of Docker Compose. By centralizing the management of networks, volumes, and containers into one configuration file, it drastically reduces deployment complexity, eliminates manual setup errors, and ensures a consistent development environment.